

Hello everyone, thank you for taking the time out of your no-doubt packed sche

My name is Matt Campbell, and as you can see on the slide, I'm one of the solution architects at Qualys, assisting our customers, and prospective customers to select, deploy and optimize Qualys products to achieve their security objectives, and advise on their security practices more generally.

This isn't what I've always done, I started my career as a developer and then application architect, then moved on to work in various system architecture roles working large scale and complex CI/CD programmes at Ericsson, one of the largest telco infrastructure OEM's.

All this to say that the process of software development, and the assurance of software quality at scale, especially the vital security quality metric that's so important for telco systems, is a topic dear to my heart and the reason I'd like to talk about the impact of AI driven development on the security of software today.

Before going any further, I would like to heavily caveat this talk. As we all know, things are moving very fast in this space, and there is a lot of uncertainty. Some of what I say, may very well be superseded by events tomorrow. In addition, a lot of the data we need to draw conclusions from is fragmented, and goes out of date quickly.

Unlike many other topics I've presented on, where extremely clear direction can be given, use this class of tool, or implement that organisational practice or standard, here the best I can do is give my opinion on what I think we as practitioners and an industry can do to mitigate the risks of deploying AI in our development programmes.

No doubt your organisations will be grappling with these issues over the coming months and years, so please do use forums like this, and others, or blog

posts, or conferences to share your findings and learnings, what works, what doesn't, so we can all progress.

So we all know there's a huge shift in the software development way of working

2

I'm sure many of you are making use of AI coding platforms and agents yourselves already, I certainly am, and they are truly remarkable.

Things that would have been hours of work, resolved in minutes. Refactors that were necessary, but just too painful, completed with a single prompt. Its very easy to become dependent on these tools.

Now, while I don't code for a living any more, I still participate in some niche software communities in my own time, and the growth in the capabilities of AI coding tools is nothing short of breathtaking.

And the speed at which this transformation is taking place is its own challenge.

Vibecoding, that's coding by prompting and AI with natural language, was announced as Collins dictionaries word of the year in November 2025, it was only coined by Andrej Karpathy one of the founders of OpenAI in February 2025, a startling metric for the sweep of the concept in a short time.

So what does this mean for software security quality?

The signs so far aren't great.

The chart is from Georgia Tech's Software Security Lab. They wrote some tooling that collects CVEs and other security advisories in Open Source repos and then tracks back to find the commit that introduced the vulnerability. It examines the commit comment and attempts to determine if the commit was AI written, no doubt missing many cases where the AI did the work, but a human pushed the commit.

There are a few really interesting trends to note here. The most obvious is that we're on what looks very much like an exponential, no surprise there, write more code with AI, add more vulnerabilities with AI.

The second is the current dominance of Claude Code, and anyone who has used that particular harness in the last 6 months will understand why, its really very good.

Now you might also look at this graph and say, well, 35 new vulnerabilities in a month over thousands of projects, doesn't seem like a big deal, at least at the moment, but authors of this research made clear that they are likely missing many CVEs introduced into products by AI that have no clear AI attribution. I wouldn't be surprised to find that its very much higher than this in reality, but we just don't have the traceability.

Adding proper traceability for AI changes in your repos is something I'll return to later in our discussion today.

The thing is, these are real security CVEs, in real repositories, out there in the wild, and more than half of them are severity critical or high.

So, given the difficulty in tracking AI introduced CVEs in the wild, we can turn to

4

Endor Labs are doing a great public service at the moment, big thanks to them, running ongoing tests using the SusVibes utility against new Harness/Model combinations as they're released and publishing the results.

SusVibes, written by researchers at Carnegie Mellon finds commits in open source projects that fixed a security vulnerability and included security tests to validate the fix. It then generates a simulated natural language spec for the change to be passed to the AI coding tool, which, starting from the previous commit, implements the change.

Then it tracks the % passes of the functional test suite and % passes of the security test suite of the AI change over hundreds of different commits.

So we see some impressive statistics from the most recent coding models, GPT-5.5 and Opus 4.7.

90% implementation accuracy for the top tier models now is truly impressive.

But things are looking far worse when it comes to security, with the top models having security test case pass figures in the low 20% when averaged over multiple attempts at the same task.

Feature competence is growing much more quickly than security competence.

This result is replicated by VeraCode's analysis programme they've been running

In this project they run 80 coding tasks, across 4 languages, with each coding task human designed to have obvious solutions that are insecure.

Then then run Veracode's own SAST tool on the generated code looking for the presence of vulnerabilities in specific CWEs, which are common weakness enumerations, or signatures of insecure coding practices, think of those as a potential CVE.

So again, we see the same striking trend as in Endor labs SusVibes runs, dramatic improvement in functional competence for the headline models, and only very slowly growing security competence.

Another interesting result which has real operational value is the delta between languages, with Java in particular demonstrating much worse security outcomes with AI coding than the other languages tested.

Veracode's hypothesis here is that the millions of lines of legacy Java code in public repositories that is not developed according to modern security best practices, so when the models are asked to develop Java they apply these older more insecure patterns.

I'm not sure about that as an explanation to be honest. Are we really saying there's more insecure Java out there than Javascript or Python? There is some support in the literature for this, but the picture isn't super clear, perhaps there's an issue here with study methodology or a 3rd confounding variable.

None the less, those of you developing in Java should take note.

6 I'd also like to quickly share this result from an IEEE conference paper presente

They've shown very clearly that security outcomes worsen if AI agents are given the task of iteratively improving code, certainly when the prompt is to improve features or efficiently, but even when the prompt is explicitly security focussed, with poor security outcomes strongly correlated with the number of autonomous iterations the AI runs.

For me this result is instructive in two ways, yes, agentic type code development is likely to increase the vulnerability count in written software, but that we should also see this same pattern in non-agentic use of AI for writing software, if the human review loop is inadequate.

--

Without the review step to keep this "security drift" in check, we're likely to see even worse security outcomes than naive reading of the previous results may suggest.

We'll return to human security review as a key control in the SDLC later in the webinar.

Ok, so we're clear that AI written code has issues, but then again, we've all bee

Nothing more humbling as developer to start saying “which idiot wrote this” only to find it was you, last year...

In august last year, researchers at the University of Naples published an analysis of over half a million code samples in Python and Java analyzing human and AI written code against various metrics, one of which was CWEs, or Common Weakness Enumerations, a taxonomy of “root causes” for software vulnerabilities.

Human written code was notably more secure than that written by the models they tested, particularly in Java, again that result with the difficulties AI has with Java, although it should be noted that more than half the issues in human written code in Python were due to CWE-754, or Improper Check for Unusual or Exceptional Conditions.

My supposition here is that the checked exceptions, forced at compile time in Java, mean that unusual or exceptional conditions are more likely to be explicitly handled and prevented.

So if we take CWE-754 out as a special case outlier, it gives a stark contrast between humans and the models.

I hope this is sufficient evidence to convince you there is a real issue here.

Now, AI tooling is not just introducing vulnerabilities into the source code itesel

Let's talk about slop squatting a term coined by Seth Larsson at the Python Software Foundation about a year ago, and refers to the practice of registering a package name likely to be hallucinated by an LLM as a vector for a supply chain attack.

The reason this is plausible is that hallucinated package names are to some extent predictable, there are patterns. Als tend to shorten or simplify package names, or merge two package names if they have related or overlapping functionality.

In the Spracklen et al paper they were able to demonstrate stability in these hallucinated package names, showing that repeated prompts with different contexted gave a strong bimodal distribution. Hallunicated package names were either highly stable, appearing every time for the same prompt, or random, not repeating at all.

This gives the attacker enough of a signal to work with. Attackers don't need to scrape massive prompt logs or brute force potential names. They can simply observe LLM behavior, identify commonly hallucinated names, and register them.

Its not on the slide, but Bar Lanyado of Lasso Security noticed AI models repeatedly hallucinating a Python package called huggingface-cli, which is a simpler version of the real package name `huggingface_hub[cli]`.

When he registered the simpler name as an experiment, he got 30,000 downloads in 3 months. His package was harmless, but others may not be.

There are also whole new classes of security vulnerabilities in the AI toolchain.

I'm only going to touch on this briefly here, as this single slide could easily be an entire webinar in itself, but the main take away is that this goes beyond the issues you'd have introducing any new tooling.

Three things make this layer structurally different from any tool category your CVE programme already covers.

[Click]

One: prompt injection is a vulnerability class, not a CVE. This is OWASP LLM01, and we'll talk about this in the context of customer facing AI in production in part two, and that's usually where its considered.

but here we're focusing on developer facing AI and what prompt injections mean for the development process.

Basically, a prompt injection is an attack making use of the way the current AI stack works. The LLM is just one part of the toolchain, it needs to be fed with a context, which is made up of things the user supplies. These are the instructions the developer provides to drive behaviour, but its also all the files files in the repo, the README, an issue you asked it to triage, a PR you asked it to review, a doc it retrieved from the web, a tool description on an MCP server, an AI rules file in the project.

[Click]

So the attack surface here isn't an exposed port or IP address, its the entire AI context, and there isn't really much tooling to so far to defend this.

We have already seen examples of this in the wild.

Some interesting examples are Github CamoLeak from mid 2025, since patched.

Camo is githubs trusted image fetching and caching platform, images here being graphics not container images or other build artifacts.

Basically, eventhough github has controls in place, to prevent data exfil, camo is a trusted domain and so by embedding prompts for github copilot in say issues or files the developer asks copilot chat to inspect, the security researcher was able to exfil sensitive data, like API keys, by embedding them in the fetch urls passed to camo that pointed to the attackers server, so the data would show up in their request logs.

Rules files are another really exposed vector for prompt injections at software build time. AI tools read files like these by default and will make the contents of this file part of the context. Store a file like this in the repo via a pull request, or shipping it in a popular template or dependency and you can get the AI to backdoor the software being developed, or any number of other downstream attacks.

[Click]

The thing with these kinds of attacks is the blast radius is now anything the AI can do, which massively increases the options for follow on exploitation.

The attacker can do anything the AI has credentials for, such as pushing code, calling MCP servers, interact with the developers machine. Once you have control of the prompt, the possibilities are huge.

You used to have a build pipeline with a few places an attacker could plant something. Now we're building agentic graphs with possibly hundreds of hops per developer per day, almost none of them instrumented and all of them exposed to this same class of vulnerabilities.

Clearly there is trouble brewing in the software development process.

10

The thing is, there is a professional discipline that has been built up painfully over decades to prevent the kinds of threats to the process we're seeing with AI tooling

Fundamentally, software development, or software engineering, is a professional practice, with codes of behaviour and the expectation of professional conduct by practitioners.

Yes, security tooling like SAST and DAST tools, CI/CD loops, test cases and so on, help to ensure security quality, but they aren't the real drivers.

The analogy for me here is the building industry. Buildings don't stay up because the building inspectors look at every weld and concrete pour.

They're safe for us because the architect, the engineer and the construction crew behaved professionally. They applied their training, followed standards and took responsibility for their work.

Yes, monitoring matters as part of a quality assurance process, but the primary reason buildings are safe is because of the professional practices of the people who designed and built them and we enter those buildings confidently because we delegate trust to those professionals.

Same for software security. We don't audit every line of code from 3rd party vendors we put into production, or decompile every binary in our toolchains. We trust that the developers made professional decisions when writing the code, that the reviewers were diligent in their oversight. We trust that dependencies were selected and tested alongside the application code and that there was sufficient determinism to have confidence that tests can tell you how software will behave when deployed.

The key word here being trust. This is the foundation the entire secure SDLC rests on.

So, now, when AI is inside every one of those trust delegations, what should be our response as organisations and as an industry as a whole?

901 what do we need to put in place tooling wise, to secure our AI SDLC?

Nothing on this slide is a new control, with the exception of prompt filtering actually, but the rest is standard.

The shift is treating the AI tools your developers run as their own asset class and managing them, properly inventoried, hardened, gated, pinned - and applying existing controls without exception.

[Click]

Asset inventory first. The old “you can’t secure what you can’t see cliché”. AI-asset and MCP-server discovery is the new shadow-IT discovery problem, just on new asset types. Every harness, every MCP server, every model endpoint, every coding assistant has to be inventoried before any other control matters. You can't govern prompt content, data-leakage, credential exposure and so on, on tools you don't even know exist.

[Click]

Endpoint controls on developer machines and agent workloads come next. Prompts leaving the developer endpoint should be inspectable; secrets in those prompts should be blocked; file context should be governed; tool calls should be allowlisted. Prompt filtering and context control tools are coming to market, open source and proprietary, so vendor select and deploy them. Data loss prevention tools should already be in place already, and many of those tools will be adding AI awareness. Dump those that don't

[Click]

Then your CI/CD tooling. Software quality gates, which should be in place already, will need some fine tuning. Its time to up the frequency for AI produced content. SAST, secret scanning, SCA on every AI-assisted commit made mandatory. Set standards for the data

stored in every commit message, tool and model versions, tools and MCP servers invoked during the code change. This is another topic that could almost be a webinar in its own right, but as an organisation, you want assess, how much information do we need now to understand the context that was active during our changes. Things like storing a hash or id that links to the full AI session stored in a DB somewhere, and other approaches. Expect rapid growth of opensource and proprietary solutions for this also.

Block hallucinated imports at CI by allowlisting against known-good package registries.

[Click]

Pin AI coding tool versions. The "harness" layer - Cursor, Copilot, Claude Code, Cline - these are the wrappers that turn LLM prediction into action on the developer's machine; treat them as third-party software with version pinning, not as continuously updating SaaS. Treat MCP servers as third-party dependencies subject to your existing controls.

There is a growing corpus of research on MCP server threats and security approaches. Another area to expect rapid development of standards and tooling.

Tools enforce rules, they don't write rules, that's up to us. This slide is my list of

12

[Click]

Everyone should have an AI Acceptable Use Policy and many of you probably already do, although many don't. The Stack Overflow Developer Survey of 2025 tells us 84 percent of developers are using AI, but only 28 percent of organisations have a formal policy. So there's the signal for Shadow AI. Now this was from June 2025, eons ago, so it will be interesting to see what that figure is this year.

[Click]

Data classification, which a data is allowed to passed to which AI tool. This depends on the both the nature of the data, is it covered under GDPR, is it commercially sensitive, and the nature of the legal framework you have with the AI provider. Without this, the AI-aware data loss prevention we mentioned on the last slide has nothing to enforce.

[Click]

Threat modelling of AI systems is most often talked about in the context of deploying production AI in your systems, but I think it applies equally well to your developer tooling. We saw previously there are completely new vulnerability modes in AI tooling, you should have a formal way of identifying and tracking these because you may need to be creative in your design of compensating controls.

[Click]

Tool Intake policies describe what a tool intake assessment looks like, what needs to be considered. Things like training on your data and data retention. Data residency, breach notifications from providers. Assessing tools and models for operational resilience and safety. What gates does a tool need to pass through to make it into your organisation?

[Click]

You want to have your AI incident response clear before it happens so you're not scrambling when it does. The AI tools and providers you select will have incidents that you will need to respond to, so be prepared for that.

[Click]

The final one to be aware of is the guidance from the UK's Information Commissioners Office stating that the "vast majority" of processing of data with AI triggers a legal requirement for a Data Protection Impact Assessment. Even if the data is not caught by GDPR, you need the assessment to document your reasoning that it is not.

Again, none of these are new behaviours, we're very used to creating policies to classify and manage our cyber security risk, we just need updated policies for these new technologies and practices.

And again, they're also not limited to what I've shown here, its up to you as practitioners to work with your organisations to have a clear look at what you need to manage in your organisation and be able to justify to your stakeholders.

The reflex instinct of a security organisation when faced with tooling that introduces

This is truly not an option with AI tooling. Not only will it adversely impact your organisations ability to compete, your developers will end run around you with shadow AI tooling, you already know and understand this.

[Click]

So, create a sanctioned tooling pathway with a weekly intake cadence. GitLab's DevSecOps 2025 survey found 49 percent of organisations run five or more AI tools. Quarterly governance bodies cannot keep up. Weekly might still be too slow but it's a start.

Second, default-allow with guardrails, not default-block. The shadow-AI problem exists because the easy path inside the company is "no". Flip that. Tools that pass intake review are immediately available to engineering. Tools that fail get a clear "here is what you'd need to fix to make this work" response, not a cold rejection.

Third, publish the yes. A visible "tools approved this week" update for dev teams does more for shadow-AI reduction than any other practice, because it removes the *reason* shadow AI is the easy path.

[Click]

Some cultural changes will be needed, not really dramatic ones, perhaps codifying more explicitly some things already in place.

No merge until reviewed by someone who did not write the prompt - if the developer who wrote the prompt is also the only person who reads the PR, the review collapses into self-review.

Pair-review for AI-generated code goes one step further - one developer who used AI plus one who did not, because the non-AI reviewer brings the

unmodified instinct of "wait, why is this here?"

Ownership remains with the human, the name on the merge owns the artefact, and reward catches alongside velocity in your performance metrics, your team-level dashboards, your engineering-leadership reviews, if they reward merge rate without rewarding catch rate, you're training the wrong reflexes in your people`.

[Click]

Some organisational changes will needed. AI compresses authorship time. It does not compress review time, in fact it expands it (there's a lot of research supporting that) So your senior engineers, who used to spend most of their time writing, now need to spend most of their time reviewing. Seniors review more, write less. Junior and mid-level engineers do more authorship - with AI assistance - and seniors apply the residual judgment AI cannot do.

Governance here is about being able to demonstrate to your stakeholders that you

14

Again nothing really new here, just applied explicitly to your AI tooling.

[Click]

It can be really helpful to align to a standard, both for you and as a signpost for your auditors. I've suggested a few which are more about production use of AI systems, rather than explicitly targeted at SDLC, but standards like this are often reasonably flexible in their implementation, and its definitely going to give you that scaffolding to build your own governance around.

[Click]

Every governance process needs an accountable owner and they need to be supported by those with an operational dog in the fight. For me this means strong collaboration between SecOps and your development organisation. Both of these teams have a big stake in the outcome of any governance process for AI tooling, make them part of the process.

[Click]

I think we all understand, governance without measurement isn't effective. Track outcomes in terms of what's happening in your SDLC. I've suggested some metrics that I think would be useful in making decisions about AI tooling in the SDLC, no doubt there are many more. Think about the data you need to understand how your decisions about AI governance are impacting your security quality metrics.

Now I said at the beginning I needed to heavily caveat today's talk due to the space

15

Perhaps this is just a moment in time and the tooling will improve so fast that everything I've talked about today will become a non issue.

[Click]

Look at aviation, modern airliners fly mostly on autopilot, about 70% of commercial flight time. Senior airline pilots, my father was one, trust the autopilot more than they trust other pilots. Dog joke.

[Click]

We touched on this stat in the beginning, at the leading AI companies, they don't write much code manually anymore. Boris Cherny, who created Claude Code, ships around 22 pull requests a day with Claude writing 100 percent of the code. Roon, the unidentified insider at OpenAI says they do not write code any more. These are real people doing real work, building the models the rest of the industry is adopting.

[Click]

Defensive tooling is improving alongside the developer tooling. AI-BOM specs are maturing. AI-aware SAST is shipping. Both sides of the equation are getting better, not just the offence.

[Click]

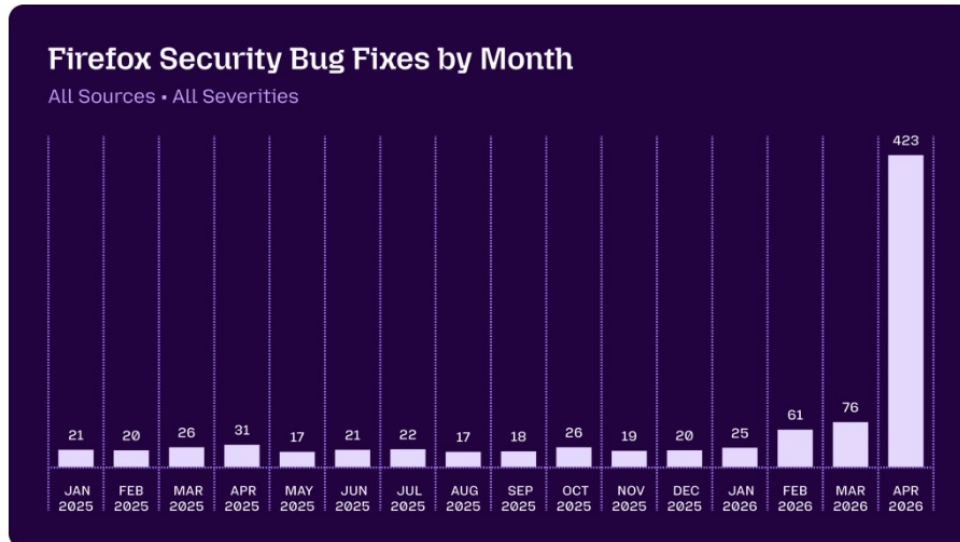
This is also the historical pattern. We used to read assembly to verify compilers. We used to manage memory by hand. Each abstraction layer was alarming at the time, and we adapted. Maybe AI-as-author is just the next layer and the problem will fade as it improves.

[Click]

Perhaps that's already happened. GPT-3.5 to GPT-4 to

GPT-5 showed enormous reasoning gains. Now we have Claude going from Opus to Mythos. The security competence flat line we identified at the start of the talk may soon change its trajectory.

Mythos is Starting to Deliver



DE-RISK YOUR BUSINESS



Indeed all indications are that it has. Thursday last week, as I was putting the final touches on this slide pack, Mozilla published the results of the security fixes that have been enabled by the early access they have to Claude Mythos.

This is a truly startling result. These are security fixes only, and they've shipped more than an order of magnitude more in a month with access to Mythos than across their historical average.

In the same way that feature competence had the startling rise over the last 18 months, the same may be about to happen to security competence with the models. If this early result bares scrutiny and AI models begin to outperform humans on security implementation, maybe even Java applications will be safe to write with AI!

Thank you very much for your time today, I hope you found this topic as interesting as I did.

If anyone does want more information on any of the topics today, or would like links to source material, please let me know, I'd be very happy to provide them and I'm of course always happy to come and speak to you about topics like these or anything else to do with cyber security.

So, now I'll hand you back to Ben for Q&A.